



O CONTROLADOR DE VOO: PAPEL HUMANO, AGENTE E VERIFICAÇÃO

Uma leitura prática sobre o desenvolvimento orientado a agentes

Heverton Eduardo Peres

ENGENHEIRO DE SOFTWARE & MAKER

Heverton Eduardo Peres

O Controlador de Voo: Papel Humano, Agente e Verificação

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

São Paulo
2026

Dados Internacionais de Catalogação na Publicação (CIP)

P512c PERES, Heverton Eduardo

O Controlador de Voo: Papel Humano, Agente e Verificação / Heverton Eduardo Peres. – São Paulo : Fábrica Agêntica de Livros, 2026.

33 p. ; 21 cm.

ISBN 978-65-00000-10-8

1. Controlador. 2. Papel. 3. Humano. 4. Agente. I. Título.

CDD 006.3

Sumário

O Controlador de Voo: Papel Humano, Agente e Verificação	6
Capítulo 2: O Controlador de Voo: Papel Humano, Agente e Verificação	7
Introdução	7
Explica	7
Ilustra	8
Técnica	9
O Contrato de Delegação como Dataclass	9
A Fila de Verificação Adversarial	10
A Árvore de Decisão de Delegação	11
O Modelo de Decisão com Três Camadas de Verificação	12
O Modelo de Escalonamento de Exceções	13
O Contraste Humano-Agente: o Que Cada Um Faz Melhor	14
O Modelo de Transferência de Autoridade	15
O Modelo de Revogação de Autoridade	16
O Modelo de Autonomia Escalonável	17
O Modelo de Matriz de Responsabilidade em YAML	17
O Comitê de Exceções como Instância Final	18
O Contrato de Delegação com Rastreabilidade	18
O Comitê de Exceções como Instância Final	19
O Registro de Decisões como Trilha de Auditoria	20
O Modelo de Análise de Capacidade do Agente	21
A Roda de Delegação em Cascata	21
A Linguagem do Contrato: Citações de Autoridade	23
O Quadro de Autoridade Visível	23
O Papel da Evidência na Decisão Humana	23
Delegar sem Abandonar	24
A Trilha de Decisões da Delegação	24
Passos para Implantar a Matriz na Sua Equipe	24
O Modelo de Contrato com Autoridade por Tarefa	25
O Limite da Autonomia Tática	25
Aplica	26
Conclusão	27
Por que este capítulo importa	27
Conceitos-chave deste capítulo	27

Checklist para aplicar hoje	28
Perguntas que você deve se fazer	28
Glossário rápido	28
O erro mais comum nesta fase	29
Um exemplo concreto para fixar	29
A rotina de quem já opera assim	29
O que não fazer: os anti-padrões mais comuns	30
Como medir o progresso na prática	30
O papel do líder neste capítulo	30
Perguntas frequentes honestas	31
Um convite para a prática deliberada	31
Próximos Passos	31

O Controlador de Voo: Papel Humano, Agente e Verificação

Uma leitura direta e prática para quem quer levar o desenvolvimento orientado a agentes a sério — sem jargão acadêmico, com exemplos aplicáveis.

Capítulo 2: O Controlador de Voo: Papel Humano, Agente e Verificação

Introdução

No Capítulo 1, você dominou a mudança estrutural do SDLC clássico para o AI-first: o artefato-mestre virou a spec executável, o custo dominante virou token e contexto, e a metáfora da torre de controle de tráfego aéreo ganhou forma. Agora chegou a hora de ocupar o posto: o que exatamente um controlador de voo de software faz, o que ele delega, e — o ponto mais delicado — o que ele **nunca** delega.

Este capítulo apresenta a matriz de papéis do SDLC AI-first em três dimensões: o humano como orquestrador e árbitro, o agente como executante, e a verificação como função separada e adversarial. Você vai aprender por que “quem escreve não valida sozinho” é a regra de ouro, e vai sair com uma ferramenta prática — um contrato de delegação — para aplicar no trabalho.

Explica

A palavra “autonomia” é a mais mal compreendida do vocabulário agêntico. Quando um agente resolve um issue do GitHub de ponta a ponta no SWE-bench, ele está sendo autônomo no nível tático: escolhe arquivos, escreve código, roda testes e corrige erros dentro de um escopo dado. Mas ele não é autônomo no nível estratégico: o problema a resolver, o critério de aceite e a definição de pronto vieram de um humano. Essa distinção entre autonomia tática e autoridade estratégica é a fundação da matriz de papéis.

A literatura de engenharia de software agêntica é clara sobre a separação de funções. Pesquisadores do estado da arte apontam que a confiabilidade de sistemas multiagentes cresce quando papéis funcionais são estritos e isolados — arquiteto não escreve o código do desenvolvedor, auditor não é o mesmo agente que produziu o artefato. No mundo físico da fábrica de

software, essa separação tem nome: segregação de funções, o mesmo princípio que impede o auditor de auditar o próprio caixa.

A verificação adversarial é a materialização desse princípio. Em vez de uma fase final de testes, a verificação torna-se uma camada contínua e independente que **tenta refutar** o trabalho do agente. O revisor não lê o código para elogiar; lê para encontrar o defeito. Essa postura — assumir que o artefato tem erro até prova em contrário — é o que transforma a revisão de código de ritual em radar.

Por que o humano permanece accountable em quase todas as fases? Porque a responsabilidade final não delega. Estudos sobre o impacto da IA generativa no desenvolvimento mostram que a delegação sem accountability produz dívida técnica silenciosa: o agente entrega rápido, o humano aprova sem ler, e o retrabalho aparece meses depois, multiplicado. O papel do humano no AI-first não é menor — é mais concentrado: ele decide menos vezes, mas decide coisas maiores.

A matriz RACI do AI-first, portanto, não é um organograma decorativo. É um contrato de autoridade que responde, para cada fase, quatro perguntas: quem executa, quem aprova, quem é consultado e quem responde. E a resposta padrão para “quem responde” é sempre o humano — inclusive quando o erro foi do agente.

Há ainda a dimensão do custo do contexto como fator de design do papel humano. Como cada turno de interação com o agente consome tokens, o humano precisa decidir **onde** gastar sua atenção: revisar diffs na Fase 5 custa pouco e evita retrabalho; revisar na produção custa muito e não evita nada. A delegação bem calibrada é, também, uma estratégia de economia de contexto.

Ilustra

A torre de controle de um aeroporto tem três funções que nunca se misturam. O controlador de voo autoriza decolagens e pousos. O piloto executa o voo. E o sistema de radar — operado por uma equipe separada, às vezes em sala diferente — monitora cada aeronave e reporta desvios. Ninguém pede ao piloto que monitore o próprio voo; o radar existe exatamente porque a percepção de quem executa é enviesada pela posição de quem executa.

Essa é a arquitetura mental do capítulo. O humano é o controlador, o agente é o piloto, e a verificação é o radar — uma função independente que observa e refuta. Quando a mesma entidade escreve e valida, você tem um piloto monitorando o próprio voo: tecnicamente possível, praticamente inútil.

[Diagrama do capítulo omitido neste formato.]

Como Comandante de Operações de Software, você notará uma sutileza: o radar também pode ser um agente. A verificação adversarial não exige humano — exige **independência**. Um segundo agente com papel de revisor, que não participou do build, tem o viés correto para refutar.

Técnica

O Contrato de Delegação como Dataclass

Vamos transformar a matriz de papéis em código. O contrato de delegação define, por fase, quem executa, quem aprova e qual artefato comprova a conclusão.

```
from dataclasses import dataclass, field
from typing import List, Optional
from enum import Enum

class AgenteTipo(Enum):
    ORQUESTRADOR = "orquestrador"
    EXECUTANTE = "executante"
    REVISOR = "revisor"

@dataclass
class ContratoDelegacao:
    fase: str
    executor: AgenteTipo
    aprovador: AgenteTipo
    accountable: str = "humano"
    artefato_prova: str = ""
    independente: bool = False

    def valida_segregacao(self) -> tuple:
        """Regra de ouro: quem executa nao pode aprovar sozinho."""
        if self.executor == self.aprovador and not self.independente:
            return False, "segregacao de funcoes violada: mesmo agente
executa e aprova"
        return True, "segregacao respeitada"

CONTRATOS = {
    "intencao": ContratoDelegacao("intencao", AgenteTipo.ORQUESTRADOR,
AgenteTipo.ORQUESTRADOR,
```

```

                                accountable="humano",
artefato_prova="intencao.md"),
    "spec": ContratoDelegacao("spec", AgenteTipo.EXECUTANTE,
AgenteTipo.ORQUESTADOR,
                                accountable="humano",
artefato_prova="spec.md"),
    "build": ContratoDelegacao("build", AgenteTipo.EXECUTANTE,
AgenteTipo.REVISOR,
                                accountable="humano", artefato_prova="diff",
independente=True),
    "verificar": ContratoDelegacao("verificar", AgenteTipo.REVISOR,
AgenteTipo.ORQUESTADOR,
                                accountable="humano",
artefato_prova="parecer.md"),
}

def checar_segregacao(fase_id: str) -> None:
    contrato = CONTRATOS[fase_id]
    ok, motivo = contrato.valida_segregacao()
    print(f"[{'OK' if ok else 'FALHA'}] {fase_id}: {motivo}")
    if not ok:
        raise SystemExit(1)

if __name__ == "__main__":
    for f in CONTRATOS:
        checar_segregacao(f)

```

Note a linha `independente=True` no contrato do build: o aprovador do build é o REVISOR, não o próprio EXECUTANTE. Essa é a codificação literal da regra de ouro do capítulo.

A Fila de Verificação Adversarial

A verificação adversarial precisa de um fluxo. O código abaixo implementa uma fila onde cada artefato produzido é encaminhado a um revisor diferente do produtor.

```

from collections import deque
from dataclasses import dataclass
from typing import List

@dataclass
class Artefato:
    id: str
    produtor: str

```

```

conteudo: str
status: str = "aguardando_revisao"

class FilaVerificacao:
    def __init__(self, revisores: List[str]) -> None:
        self.fila: deque = deque()
        self.revisores = revisores
        self.pareceres = []

    def adicionar(self, artefato: Artefato) -> None:
        if len(self.revisores) == 1 and self.revisores[0] == artefato.produtores:
            raise ValueError("segregacao violada: sem revisor independente disponivel")
        self.fila.append(artefato)

    def revisar(self) -> None:
        while self.fila:
            artefato = self.fila.popleft()
            revisor = next(
                (r for r in self.revisores if r != artefato.produtores),
                None)

            if revisor is None:
                raise RuntimeError("nenhum revisor independente")
            self.pareceres.append(
                f"{artefato.id}: revisor={revisor} status={refutacao_em_andamento}")
        return None

fila = FilaVerificacao(revisores=["agente-revisor", "revisor-humano"])
fila.adicionar(Artefato("spec-v1", "agente-redator", "escopo e criterios"))
fila.revisar()
print("\n".join(fila.pareceres))

```

A Árvore de Decisão de Delegação

Quando a delegação é possível? A resposta operacional é uma árvore de decisão que o orquestrador consulta antes de cada despacho. O código abaixo implementa a árvore — ela decide, com base no tipo de decisão, se o trabalho vai para o agente, para o revisor ou para o humano.

```

from dataclasses import dataclass
from enum import Enum

```

```

class Destino(Enum):
    AGENTE = "agente"
    REVISOR = "revisor"
    HUMANO = "humano"

@dataclass
class DecisaoDelegacao:
    tarefa: str
    risco: str          # baixo | medio | alto
    reversivel: bool
    requer_contrato: bool

    def destino(self) -> Destino:
        if self.risco == "alto" or not self.reversivel:
            return Destino.HUMANO
        if self.requer_contrato and not self.reversivel:
            return Destino.HUMANO
        if self.risco == "medio":
            return Destino.REVISOR
        return Destino.AGENTE

DECISOES = [
    DecisaoDelegacao("gerar_documentacao", "baixo", True, False),
    DecisaoDelegacao("implementar_endpoint", "medio", True, True),
    DecisaoDelegacao("mudar_schema_banco", "alto", False, True),
    DecisaoDelegacao("refatorar_modulo_legacy", "medio", False, True),
]

if __name__ == "__main__":
    for d in DECISOES:
        print(f"{d.tarefa:<28} -> {d.destino().value}")

```

O resultado é previsível e auditável: mudança de schema (alto risco, irreversível) nunca vai para o agente direto; documentação (baixo risco, reversível) vai sem cerimônia. A árvore transforma a intuição de delegação em regra executável.

O Modelo de Decisão com Três Camadas de Verificação

A matriz de papéis não existe no vácuo — ela se materializa no modelo de decisão que define, para cada artefato, quem produz, quem verifica e quem arbitra. O código abaixo implementa o modelo completo de três camadas:

```

from dataclasses import dataclass
from typing import List

@dataclass
class Camada:
    nome: str
    funcao: str
    autonomo: bool

CAMADAS_VERIFICACAO = [
    Camada("maquina", "typecheck, lint, testes", True),
    Camada("adversarial", "revisor independente que refuta", True),
    Camada("humana", "decisao de merge e arbitragem", False),
]

def modelo_decisao(artefato: str, mudanca_contrato: bool = False) -> List[str]:
    """Retorna a sequencia de verificacoes obrigatorias para um
    artefato."""
    sequencia = [f"{c.nome}: {c.funcao}" for c in CAMADAS_VERIFICACAO]
    if mudanca_contrato:
        sequencia.append("humana-extra: revisao de impacto de contrato")
    return sequencia

for passo in modelo_decisao("migracao_schema", mudanca_contrato=True):
    print(f"    -> {passo}")

```

O modelo de decisão é o coração operacional do Capítulo 2: a sequência de verificações é previsível, auditável e idêntica para todos os artefatos da mesma classe — a régua da torre aplicada a cada voo.

O Modelo de Escalonamento de Exceções

Nem toda decisão pode ser delegada, e o agente precisa saber quando subir a decisão. O modelo abaixo classifica a exceção em três níveis de escalonamento — tático, gerencial e estratégico — e roteia cada uma para o nível certo:

```

class Escalonador:
    NIVEIS = {'tatico': 1, 'gerencial': 2, 'estrategico': 3}

    def __init__(self):

```

```

self.decisoes = []

def escalar(self, decisao, impacto, irreversivel):
    nivel = 'tatico'
    if impacto == 'alto':
        nivel = 'gerencial'
    if irreversivel or impacto == 'critico':
        nivel = 'estrategico'
    self.decisoes.append({'decisao': decisao, 'nivel': nivel})
    return {'decisao': decisao, 'nivel': nivel, 'nivel_numero':
self.NIVEIS[nivel]}

e = Escalonador()
print(e.escalar('escolher nome de variavel', 'baixo', False))
print(e.escalar('mudar schema do banco', 'alto', True))

```

A tabela de escalonamento é o que evita o pior dos dois mundos: agente que decide demais (risco) ou agente que pergunta demais (paralisia). Decisão de baixo impacto e reversível fica no nível tático — o agente decide e registra. Decisão de alto impacto fica no gerencial, com contexto. Decisão irreversível ou crítica sobe ao estratégico, onde está o comandante. O registro de todas as escaladas cria o padrão de onde a organização realmente toma decisões.

O Contraste Humano-Agente: o Que Cada Um Faz Melhor

A matriz de papéis se torna mais nítida quando confrontamos as forças relativas. O quadro abaixo é o instrumento de calibração — o que delegar sem medo, o que delegar com supervisão e o que nunca delegar:

Tarefa	Agente faz melhor?	Humano faz melhor?	Decisão
Escrever testes de unidade	Sim (rápido, volumoso)	—	Delegar
Explorar código legado	Sim (varredura massiva)	—	Delegar via subagente
Definir critérios de aceite	—	Sim (conhece o negócio)	Humano
Arbitrar trade-offs de design	—	Sim (contexto e histórico)	Humano
Detectar regressão de contrato	Sim (CI é implacável)	—	Delegar à máquina
Responder por incidente	—	Sim (accountability)	Humano

A calibração não é sobre competência apenas — é sobre risco. O agente pode redigir um critério de aceite plausível, mas o critério errado custa o ciclo inteiro; o humano pode revisar cem testes, mas o volume é ineficiente. A régua é: volume e mecânica delegam; julgamento e responsabilidade permanecem.

O Modelo de Transferência de Autoridade

A autoridade não é transferida de uma vez — é transferida em etapas com critérios. O modelo abaixo define quando uma decisão pode subir ou descer de nível:

Decisão	Nível atual	Critério para subir	Critério para descer
Escopo	Humano	Impacto alto/irreversível	Rotina documentada
Merge	Humano	Contrato alterado	Mudança trivial com radar verde
Teste	Agente	Cobertura de borda baixa	Matriz completa
Skill	Revisor	Taxa de sucesso cai	Maturidade comprovada

O modelo de transferência é o contrato dinâmico de autoridade: os níveis não são fixos — mudam com o desempenho comprovado. A autoridade desce quando a evidência sustenta; sobe quando o risco cresce.

O Modelo de Revogação de Autoridade

Autoridade concedida precisa de revogação possível e automática. O modelo abaixo revoga a delegação quando o agente viola o contrato — excedendo o escopo ou deixando de registrar evidência:

```
class ContratoDeAutoridade:
    def __init__(self, id, escopo, exige_evidencia):
        self.id = id
        self.escopo = escopo
        self.exige_evidencia = exige_evidencia
        self.ativa = True
        self.violacoes = []

    def executar(self, acao, evidencia):
        if not self.ativa:
            return {'status': 'bloqueada', 'motivo': 'autoridade revogada'}
        if acao not in self.escopo:
            self.violacoes.append({'tipo': 'fora_de_escopo', 'acao': acao})
            self.ativa = False
            return {'status': 'revogada', 'motivo': 'acao fora do escopo'}
        if self.exige_evidencia and not evidencia:
            self.violacoes.append({'tipo': 'sem_evidencia', 'acao': acao})
            self.ativa = False
            return {'status': 'revogada', 'motivo': 'acao sem evidencia'}
        return {'status': 'ok', 'acao': acao}

c = ContratoDeAutoridade('A-5', {'editar_arquivos', 'rodar_teste'},
exige_evidencia=True)
print(c.executar('deletar_banco', evidencia=''))
print(c.executar('editar_arquivos', evidencia='teste_ok.txt'))
```

A revogação automática é o freio de emergência da delegação: quando o agente tenta ação fora do escopo ou age sem evidência, a autoridade morre na hora e o incidente fica registrado para o debriefing. O humano não precisa vigiar em tempo real — o contrato vigia. E o padrão de violações, acumulado ao longo de semanas, alimenta a decisão de conceder ou negar a próxima delegação.

O Modelo de Autonomia Escalonável

A autonomia do agente não é binária — é escalável. O modelo abaixo define níveis de autonomia por classe de tarefa, com a autoridade correspondente:

Nível	Autonomia do agente	Supervisão humana	Exemplo
A1	Executa com instrução detalhada	Revisão completa	Teste de unidade
A2	Executa com objetivo claro	Revisão por amostra	Feature isolada
A3	Escolhe a abordagem	Revisão de contrato	Refatoração
A4	Opera o fluxo da fase	Supervisão por exceção	Pipeline de release
A5	Opera o ciclo inteiro	Arbitragem de conflito	Maturidade L5

O modelo de autonomia escalável é o dial da delegação: cada tarefa recebe o nível de autonomia que o risco justifica — e o nível é declarado no contrato de delegação. O agente que opera a A5 sem o contrato correspondente é uma violação do modelo, não uma evolução dele.

O Modelo de Matriz de Responsabilidade em YAML

A matriz RACI do AI-first pode ser declarada em formato de máquina — o mesmo YAML que a esteira consome para validar a segregação de funções. O modelo abaixo é a matriz canônica:

```
matriz_raci:
  fases:
    intencao:
      humano_responsable: true
      agente_responsable: false
      humano_accountable: true
      regra: "agente pode provocar (grill), nunca decidir"
    spec:
      humano_responsable: false
      agente_responsable: true
      humano_accountable: true
      regra: "agente redige; humano aprova; sem aprovacao, sem build"
    build:
      humano_responsable: false
      agente_responsable: true
      humano_accountable: true
      regra: "segregacao: produtor != aprovador"
```

```

verificar:
  humano_responsable: false
  agente_responsable: true
  humano_accountable: true
  regra: "revisor independente refuta com evidencia"
entregar:
  humano_responsable: true
  agente_responsable: false
  humano_accountable: true
  regra: "release reproduzível; humano autoriza estagios"

```

A matriz em YAML é o contrato executável de responsabilidade: a esteira a consulta antes de cada transição e bloqueia qualquer desvio da regra declarada — a segregação vira código, não intenção.

O Comitê de Exceções como Instância Final

Nenhuma matriz cobre todos os casos — e é por isso que o Comandante tem um comitê de exceções: a instância mínima de humanos que arbitra quando máquina e agente discordam ou quando o caso não está no contrato. O fluxo de escalação é simples:

1. O caso chega ao comitê com a evidência das camadas (nunca só a narrativa).
2. O comitê decide com base no contrato — ou atualiza o contrato se a lacuna for real.
3. A decisão vira precedente registrado no banco de decisões (o registro do Capítulo 2).
4. O precedente alimenta a revisão da spec na Fase 8.

O comitê não é burocracia — é a válvula de segurança do contrato: quando o processo não cobre o caso, a exceção é decidida por humanos com evidência, e o aprendizado volta para o processo.

O Contrato de Delegação com Rastreabilidade

Um contrato de delegação sem rastreabilidade é uma promessa vazia. O modelo abaixo adiciona ao dataclass de delegação um histórico de versões e a ligação com a evidência que justificou cada delegação:

```

from dataclasses import dataclass, field
from datetime import datetime

@dataclass
class DelegacaoRastreavel:
    id: str
    tarefa: str
    agente: str
    autoridade: str

```

```

evidencia: str
versao: int = 1
historico: list = field(default_factory=list)

def alterar(self, campo, valor, motivo, autor):
    self.historico.append({
        'versao': self.versao,
        'campo': campo,
        'valor_antigo': getattr(self, campo),
        'valor_novo': valor,
        'motivo': motivo,
        'autor': autor,
        'quando': datetime.now().isoformat(),
    })
    setattr(self, campo, valor)
    self.versao += 1

def trilha(self):
    return {'id': self.id, 'versao_atual': self.versao, 'historico':
self.historico}

d = DelegacaoRastreavel('D-17', 'implementar cobranca', 'agente-b',
'parcial', 'spec_executavel.md#R12')
d.alterar('autoridade', 'plena', 'incidentes zerados por 30 dias',
'comandante')
print(d.trilha())

```

Cada alteração de autoridade fica registrada com motivo e autor — se um dia a delegação plena precisar ser questionada, a trilha mostra exatamente quando, por quê e com base em qual evidência ela foi concedida. Isso transforma a delegação de um ato administrativo em um fato registrado, auditável e reversível.

O Comitê de Exceções como Instância Final

Quando o agente encontra uma tarefa fora do seu contrato, quem decide? O modelo abaixo encaminha a exceção para um comitê humano, mas exige que a solicitação venha com contexto estruturado — sem ele, a exceção nem entra na fila:

```

class ComiteDeExcecoes:
    def __init__(self):
        self.fila = []

    def solicitar(self, agente, tarefa, motivo, impacto, alternativa):
        if not (motivo and impacto):
            return {'status': 'rejeitada', 'motivo': 'contexto incompleto'}

```

```

        item = {'agente': agente, 'tarefa': tarefa, 'motivo': motivo,
'impacto': impacto, 'alternativa': alternativa}
        self.fila.append(item)
        return {'status': 'na_fila', 'posicao': len(self.fila)}

    def decidir(self, posicao, decisao, justificativa):
        item = self.fila[posicao - 1]
        item['decisao'] = decisao
        item['justificativa'] = justificativa
        return item

comite = ComiteDeExcecoes()
print(comite.solicitar('agente-b', 'acessar producao', 'precisa do banco
real', 'bloqueia entrega', 'usar staging'))

```

A exigência de contexto estruturado tem efeito duplo: reduz o ruído (o comitê só vê exceções bem formuladas) e educa o agente (a cada tentativa de exceção, ele aprende o formato do que a organização considera informação suficiente para decidir). Com o tempo, muitas exceções param de existir porque o agente aprende a resolvê-las dentro do contrato.

O Registro de Decisões como Trilha de Auditoria

A responsabilidade humana precisa de trilha. O registro de decisões — quem aprovou o quê, com base em qual evidência — é o artefato que transforma accountability em dado:

```

{
  "decisoes": [
    {
      "id": "DEC-042",
      "data": "2026-07-30",
      "decisor": "lead-plataforma",
      "tipo": "autorizar_merge",
      "artefato": "feature/cupons-v2",
      "evidencia": ["saida_ci.txt", "parecer_adversarial.md"],
      "escalado_de": "agente-revisor",
      "observacao": "caso de borda cupom acumulativo coberto por teste
canonico"
    }
  ]
}

```

O registro é a caixa-preta da autoridade: quando um incidente acontece, a primeira pergunta — “quem autorizou e com que evidência?” — tem resposta imediata e objetiva. Sem trilha, a accountability é narrativa; com trilha, é auditoria.

O Modelo de Análise de Capacidade do Agente

Antes de delegar, o comandante precisa saber o que o agente é capaz de fazer hoje. O modelo abaixo mantém um perfil de capacidades por agente, atualizado a cada tarefa concluída com sucesso ou falha:

```
class PerfilDeCapacidade:
    def __init__(self, agente):
        self.agente = agente
        self.capacidades = {}

    def registrar_tarefa(self, tipo, resultado):
        cap = self.capacidades.setdefault(tipo, {'sucessos': 0, 'falhas': 0})

        if resultado == 'sucesso':
            cap['sucessos'] += 1
        else:
            cap['falhas'] += 1

    def confianca(self, tipo):
        cap = self.capacidades.get(tipo, {'sucessos': 0, 'falhas': 0})
        total = cap['sucessos'] + cap['falhas']
        if total == 0:
            return 0.5 # desconhecido
        return cap['sucessos'] / total

    def pode_delegar(self, tipo, limiar=0.8):
        return self.confianca(tipo) >= limiar

perfil = PerfilDeCapacidade('agente-b')
for resultado in ['sucesso'] * 12 + ['falha'] * 1:
    perfil.registrar_tarefa('refatoracao', resultado)
print(perfil.confianca('refatoracao')) # 0.92
print(perfil.pode_delegar('refatoracao')) # True
```

A confiança medida substitui a confiança intuitiva: o agente não recebe autonomia plena em um tipo de tarefa porque “parece capaz”, mas porque o histórico de evidências demonstra. O limiar é configurável por tipo de tarefa — tarefas de baixo risco podem usar limiar 0.7, tarefas que tocam produção exigem 0.95. Quando a confiança cai abaixo do limiar após uma sequência de falhas, o contrato de delegação é revogado automaticamente.

A Roda de Delegação em Cascata

A delegação não acontece em um único nível — ela desce em cascata. O Comandante delega a um agente; o agente pode delegar partes a subagentes; e a verificação separa cada nível. A cascata precisa de controle, e o controle é o registro de toda a árvore de delegação:

```

from dataclasses import dataclass, field
from typing import List, Optional

@dataclass
class NoDelegacao:
    tarefa: str
    agente: str
    pai: Optional[str] = None
    filhos: List["NoDelegacao"] = field(default_factory=list)

    def profundidade(self) -> int:
        if not self.pai:
            return 0
        return 1

    def caminho(self) -> str:
        return f"{self.pai} -> {self.agente}" if self.pai else self.agente

def montar_arvore_delegacao() -> NoDelegacao:
    raiz = NoDelegacao("feature pagamentos", "orquestrador")
    modulo = NoDelegacao("modulo fatura", "agente-pagamentos",
pai="orquestrador")
    interface = NoDelegacao("interface", "subagente-interface",
pai="agente-pagamentos")
    testes = NoDelegacao("testes de aceite", "subagente-testes",
pai="agente-pagamentos")
    modulo.filhos = [interface, testes]
    raiz.filhos = [modulo]
    return raiz

def listar_arvore(no: NoDelegacao) -> list:
    saida = [no.caminho()]
    for filho in no.filhos:
        saida.extend(listar_arvore(filho))
    return saida

arvore = montar_arvore_delegacao()
for linha in listar_arvore(arvore):
    print(f" {linha}")

```

A árvore registrada é o mapa de responsabilidade: se o artefato da interface falhar, o caminho de auditoria sabe exatamente quem produziu, sob qual pai, em qual nível.

A Linguagem do Contrato: Citações de Autoridade

O contrato de delegação também define a linguagem da autoridade — as expressões que separam uma decisão humana de uma ação agêntica:

Expressão	Significado	Quem emite
“Autorizado”	Decisão final com responsabilidade	Humano
“Recomendado”	Proposta sem autoridade final	Agente
“Evidência:”	Dado que sustenta uma afirmação	Qualquer um, com registro
“Pendente:”	Item que bloqueia avanço	Esteira
“Escalado”	Decisão subiu de nível	Agente → Humano

A linguagem uniforme evita o mal-entendido mais caro do AI-first: o agente que trata “recomendado” como “autorizado” e age além do mandato. O vocabulário do contrato é a fraseologia da torre — sem ambiguidade entre setores.

O Quadro de Autoridade Visível

O contrato de autoridade precisa ser visível para o time inteiro — não um arquivo esquecido. Um quadro simples, versionado no repositório, responde em segundos quem pode o quê:

Decisão	Executor	Aprovador	Accountable	Delegável?
Escrever teste	Agente	Revisor agêntico	Humano	Sim
Definir escopo	Humano	Humano	Humano	Não
Escolher arquivo	Agente	Revisor	Humano	Sim
Autorizar merge	Humano	—	Humano	Não
Criar skill	Agente	Revisor humano	Humano	Parcial

O quadro é o mapa de autoridade da torre: cada controlador sabe o próprio raio de ação sem ambiguidade — e o auditor sabe onde procurar quando o processo falha.

O Papel da Evidência na Decisão Humana

A autoridade estratégica do humano depende de um instrumento que ainda não formalizamos: a evidência. Decisões de merge tomadas sem registro de evidência são decisões de fé — e a fé não escala quando dezenas de mudanças agênticas cruzam o radar por semana. Organizações

que lideram o movimento agêntico institucionalizaram a porta de evidência: nenhuma mudança avança sem o output registrado das camadas de verificação, e o parecer humano arbitra apenas as exceções que o radar não resolveu sozinho. Essa separação — máquina filtra, agente refuta, humano arbitra — é o que permite escalar a delegação sem escalar o risco.

Delegar sem Abandonar

A distinção entre autonomia tática e autoridade estratégica também define o estilo de delegação do Comandante. Delegar não é abandonar: é entregar a execução e manter a supervisão do contrato. O agente que roda uma suíte inteira de testes e volta com o relatório é útil; o agente que decide sozinho o escopo da próxima iteração é perigoso. A literatura de agentes de engenharia de software reforça que os melhores resultados aparecem quando o humano define a intenção e os critérios, e o agente resolve o espaço entre os dois. Ferramentas de revisão assistida já automatizam a análise de codebases gigantes, mas o julgamento sobre o que vale a pena mudar permanece humano.

A Trilha de Decisões da Delegação

Cada delegação importante deixa trilha — e a trilha é pública. O padrão de registro: quem delegou, para quem, com qual escopo, baseado em qual evidência, com qual nível de autoridade e quando. A trilha pública muda o comportamento de quem delega: a decisão de conceder autoridade plena a um agente fica visível para o time inteiro, e o delegador responde por ela no debriefing. A trilha também é o material de treinamento da organização — os padrões de delegação que funcionaram (e os que quebraram) estão todos registrados para o próximo comandante.

Passos para Implantar a Matriz na Sua Equipe

1. **Liste as fases** do seu ciclo de vida (reutilize o inventário do Capítulo 1).
2. **Atribua a cada fase** um executor, um aprovador e um accountable, usando o contrato de delegação acima.
3. **Force a segregação**: crie um teste automatizado que falha se o mesmo agente aparece como executor e aprovador na mesma fase.
4. **Distinga o radar humano do radar agêntico**: defina quais fases exigem revisor humano (spec, merge) e quais aceitam revisor agêntico (build preliminar).
5. **Registre cada parecer** com evidência (output de comando, diff, métrica) — afirmação sem evidência não é parecer.

O Modelo de Contrato com Autoridade por Tarefa

A autoridade não precisa ser global — pode ser concedida por tarefa. O modelo abaixo define permissões granulares que o agente usa para saber exatamente o que pode fazer:

```
class AutoridadePorTarefa:
    def __init__(self):
        self.permissoes = {}

    def conceder(self, tarefa, acoes):
        self.permissoes[tarefa] = set(acoes)

    def pode(self, tarefa, acao):
        return acao in self.permissoes.get(tarefa, set())

a = AutoridadePorTarefa()
a.conceder('refatorar-modulo', ['ler', 'editar', 'testar'])
print(a.pode('refatorar-modulo', 'editar'))
print(a.pode('refatorar-modulo', 'deployar'))
```

A autoridade por tarefa é a versão fina do contrato de delegação: o agente pode editar e testar na tarefa de refatoração, mas não pode deployar em nenhuma. Conceder permissões granulares reduz o risco sem aumentar a burocracia — a pergunta não é “o agente é confiável?” mas “para esta tarefa, quais ações fazem sentido?”.

O Limite da Autonomia Tática

Nem toda decisão pode ser delegada. O trecho abaixo lista decisões que permanecem estratégicas — bloqueadas por padrão para o agente:

```
DECISOES_ESTRATEGICAS = {
    "aprovar_escopo": {"nivel": "estrategico", "delegavel": False},
    "definir_done": {"nivel": "estrategico", "delegavel": False},
    "autorizar_merge": {"nivel": "estrategico", "delegavel": False},
    "escolher_arquivo": {"nivel": "tatico", "delegavel": True},
    "escrever_teste": {"nivel": "tatico", "delegavel": True},
    "rodar_lint": {"nivel": "tatico", "delegavel": True},
}

def pode_delegar(decisao: str) -> bool:
    return DECISOES_ESTRATEGICAS.get(decisao, {}).get("delegavel", False)

if __name__ == "__main__":
```

```

for decisao in DECIS0ES_ESTRATEGICAS:
    nivel = "delegavel" if pode_delegar(decisao) else "humano
obrigatorio"
    print(f"{decisao:<20} -> {nivel}")

```

Essa lista é o coração do contrato de autoridade: autonomia tática ampla, autoridade estratégica concentrada.

Aplica

Cena real, em segunda pessoa. Você é o tech lead de uma plataforma de e-commerce. Seu time adotou um agente de IA para implementar features, e tudo corre bem — até a feature de cupom de desconto. O agente implementa a lógica, roda os testes locais e abre o pull request. Você, ocupado com uma reunião, aprova o PR com uma olhada rápida. Na sexta-feira, um cupom acumulativo de 30% mais 30% vaza e gera prejuízo de R\$ 80 mil em uma hora.

O erro tem dois andares. Primeiro andar: o agente implementou sozinho e se auto-validou — os testes que ele rodou foram os testes que ele mesmo escreveu, para o cenário que ele mesmo imaginou. Segundo andar: você aprovou sem revisão adversarial — o mesmo PR que o agente escreveu foi lido pelo mesmo fluxo que o gerou, sem nenhum par de olhos com postura de refutação.

O diagnóstico ligado à teoria: a segregação de funções foi violada nos dois níveis. O produtor validou a própria produção (violação da regra de ouro), e o aprovador humano atuou como carimbo, não como radar (violação da autoridade estratégica).

A correção prática:

1. **Configure um revisor agêntico independente** que rode automaticamente em todo PR do agente, com postura explícita de refutação: procurar o cenário que quebra, não o que funciona.
2. **Exija que o PR do agente inclua** os casos de borda que ele testou e os que ele **não** testou — uma declaração de limites, não só de sucesso.
3. **Reserve 15 minutos por PR** para a revisão humana focada apenas nas decisões estratégicas: escopo, critérios de aceite e impacto em produção. Todo o resto é do radar agêntico.
4. **Trate o incidente como insumo de aprendizado**: o caso do cupom acumulativo vira um caso de teste canônico e uma regra de negócio explícita na spec, não apenas um hotfix.

Armadilhas que você encontrará no caminho: achar que a revisão agêntica substitui a humana (substitui em volume, não em autoridade); aceitar pareceres sem evidência (“revisei e está ok” sem output de comando); e permitir que o mesmo agente escreva o código e a suíte de testes da mesma feature sem um terceiro olhar.

Conclusão

Neste voo, você assumiu o posto de controlador. Recapitulando os três marcos: primeiro, a distinção entre autonomia tática (delegável ao agente) e autoridade estratégica (concentrada no humano); segundo, a segregação de funções como princípio — quem escreve não valida sozinho, materializada em código no contrato de delegação e na fila de verificação; terceiro, a verificação adversarial como radar independente, que pode ser humana ou agêntica, desde que independente.

Como desafio, implemente o contrato de delegação do seu time em código e rode o teste de segregação: você vai descobrir pelo menos uma fase onde o produtor é também o aprovador.

No próximo capítulo, você vai escrever o primeiro artefato do novo ciclo: transformar intenção vaga em spec executável — o plano de voo que autoriza a decolagem.

Por que este capítulo importa

Se você chegou até aqui, já percebeu que o controlador de voo: papel humano, agente e verificação. Este capítulo — *Capítulo 2: O Controlador de Voo: Papel Humano, Agente e Verificação* — é um convite para parar de usar IA como ferramenta de autocomplete e começar a tratá-la como parte estrutural do seu processo de desenvolvimento. A diferença entre as duas posturas é exatamente o que separa quem apenas acelera o que já fazia de quem repensa o que é possível.

Vamos explorar isso com exemplos práticos, código real e um passo a passo que você pode aplicar ainda hoje, sem esperar por infraestrutura nova ou aprovação de comitê. A ideia é simples: cada seção termina com algo que você pode executar em menos de uma hora.

Conceitos-chave deste capítulo

- **O contrato antes da execução:** antes de deixar qualquer agente trabalhar, defina o que significa “feito” em termos verificáveis.
- **Evidência antes de afirmação:** o que não pode ser verificado não pode ser delegado com segurança.
- **Aprendizado contínuo:** cada entrega, boa ou ruim, é matéria-prima para o próximo ciclo.

Esses três conceitos aparecem, de formas diferentes, em todas as seções a seguir. Mantê-los em mente enquanto você lê vai transformar exemplos isolados em um padrão que você reconhece na sua própria rotina.

Checklist para aplicar hoje

1. Escolha uma tarefa pequena e bem definida do seu backlog.
2. Escreva o critério de aceite em uma frase verificável.
3. Delegue a execução a um agente, mantendo a verificação em suas mãos.
4. Registre o que funcionou e o que não funcionou.
5. Transforme o aprendizado em um procedimento reutilizável.

Se você fizer apenas o primeiro item, já estará à frente da maioria das equipes — que continua discutindo IA em reuniões sem nunca definir o que quer que ela faça.

Perguntas que você deve se fazer

1. Qual fase do meu processo consome mais tempo hoje — e por quê?
2. O que eu delegaria a um agente amanhã se tivesse certeza de que o resultado seria verificado?
3. Qual informação eu poderia registrar hoje que tornaria a próxima iteração mais barata?
4. Quem no meu time revisa o trabalho de quem — e com qual critério?
5. O que eu faria se o custo de cada tentativa caísse para quase zero?

Essas perguntas não têm resposta certa, mas têm uma propriedade em comum: elas forçam você a sair da conversa abstrata sobre IA e entrar no terreno do seu processo real. E é exatamente nesse terreno que o SDLC AI-first produz resultado.

Glossário rápido

- **Agente:** programa que usa um modelo de linguagem para planejar e executar tarefas com acesso a ferramentas.
- **Harness:** a camada que conecta o agente ao ambiente — arquivos, comandos, testes e regras.
- **Spec executável:** especificação cujos critérios podem ser verificados por máquina.
- **Verificação adversarial:** camada que refuta o trabalho produzido, em vez de apenas confirmá-lo.
- **Contexto:** a janela de informação que o modelo enxerga a cada passo — o recurso mais caro do ciclo.

Dominar esses cinco termos é suficiente para acompanhar qualquer discussão séria sobre desenvolvimento orientado a agentes.

O erro mais comum nesta fase

A maioria das equipes comete o mesmo erro: adota a ferramenta e mantém o processo. O agente entra no fluxo como um autocomplete sofisticado, e todo o potencial de transformação se perde em pequenas conveniências. O antídoto é simples e desconfortável: mude o processo primeiro, depois traga a ferramenta. Defina o contrato, o critério de aceite e a verificação antes de permitir que o agente produza em escala. É contra intuitivo, mas é o que separa as equipes que capturam valor das que apenas geram volume.

Um exemplo concreto para fixar

Imagine uma pequena feature de faturamento. No fluxo tradicional, um desenvolvedor recebe a tarefa, interpreta a intenção, escreve o código e um revisor confia na leitura. No fluxo orientado a agentes, a mesma feature começa com uma frase verificável: “o valor total deve considerar o desconto aplicado antes dos impostos”. O agente implementa, os testes verificam a regra, e o humano revisa a evidência — não o código linha a linha, mas o comportamento observado. Perceba o deslocamento: o humano deixa de ler tudo para auditar o essencial, e o agente deixa de adivinhar para executar contra um critério. É essa troca que o restante do livro explora em profundidade.

A rotina de quem já opera assim

Uma semana de trabalho em uma equipe que já adotou o ciclo orientado a agentes não parece uma revolução — parece um fluxo calmo e bem definido. Na segunda-feira, a especificação da semana é revisada em uma reunião curta: cada critério de aceite é lido em voz alta e qualquer ambiguidade é resolvida antes de tocar em código. Na terça, os agentes executam as tarefas em isolamento, enquanto os humanos revisam a arquitetura e os contratos. Na quarta, a verificação roda: testes, revisão adversarial e a decisão de merge apoiada em evidência. Na quinta, o que passou vai para produção em canário, com observabilidade ligada. Na sexta, o debriefing transforma os incidentes da semana em lições e skills. Nenhum dia é heroico; todos os dias são previsíveis. E é exatamente essa previsibilidade — não a velocidade máxima — que define a alta performance.

O que não fazer: os anti-padrões mais comuns

Se você quer destruir o valor do desenvolvimento orientado a agentes, aqui estão as receitas mais eficientes. Primeiro, o prompt-and-pray: gere o código, olhe por cima, e peça desculpas quando quebrar. Funciona em demos, falha em produção. Segundo, a spec decorativa: escreva documentos longos que ninguém verifica e que o agente não consegue executar — o pior dos dois mundos. Terceiro, a auto-verificação: deixe que quem escreveu valide o próprio trabalho, sem revisor independente; é a forma mais rápida de transformar confiança em acidente. Quarto, a delegação sem observabilidade: conceda autonomia sem instrumentar o comportamento. Todos esses padrões têm uma origem comum — a pressa em capturar o ganho sem construir o controle. E todos têm o mesmo antídoto: contrato antes de execução, evidência antes de afirmação, e revisão independente em toda entrega.

Como medir o progresso na prática

Uma dúvida legítima é: como saber se a adoção está dando certo? Métricas tradicionais de velocidade podem enganar — um time pode entregar mais rápido e acumular dívida técnica invisível. O indicador mais confiável no ciclo orientado a agentes é a estabilidade: quantos incidentes em produção, quanto tempo de retrabalho, quantas correções de emergência. Um segundo indicador é o custo de contexto: quantos tokens cada fase consome, e onde o desperdício se concentra. Um terceiro é a taxa de aceite na primeira verificação: se os agentes precisam de muitas rodadas de refutação, o contrato está fraco — o problema não é o agente, é a spec. Com esses três números na mesa, a conversa de progresso deixa de ser anedótica e vira análise de processo.

O papel do líder neste capítulo

Nada do que este capítulo descreve acontece por acaso — alguém precisa criar as condições para que o processo exista. Esse alguém é o líder técnico, o líder de equipe ou o arquiteto que decidiu tratar o ciclo orientado a agentes como uma mudança de processo, não como a instalação de uma ferramenta. O trabalho do líder aqui tem quatro frentes. A primeira é a modelagem do contrato: garantir que cada fase tem entrada, saída e critério definidos. A segunda é a calibragem da confiança: decidir o que pode ser delegado e o que exige decisão humana, e documentar essa decisão. A terceira é a defesa do tempo de verificação: em uma cultura que celebra velocidade, o líder precisa defender o orçamento de revisão como quem defende o seguro do prédio. A quarta

é o exemplo: o líder que pede evidência antes de afirmar, em toda reunião, ensina mais do que qualquer documento de processo.

Perguntas frequentes honestas

P: Isso não vai tirar o emprego dos desenvolvedores? R: A história do ciclo de vida nunca foi sobre menos trabalho humano, mas sobre trabalho mais valioso. O que muda é a natureza da tarefa: escrever código repetitivo deixa de ser o centro, e a especificação, a verificação e o desenho de contratos ocupam o lugar. P: Precisamos de um time de especialistas em IA para começar? R: Não. Precisa-se de disciplina de processo e de vontade de medir. As ferramentas evoluem rápido; o processo é o que permanece. P: E se o agente produzir código que ninguém entende? R: Essa é a pergunta certa — e a resposta é a verificação: se o código passa nos testes, na revisão adversarial e na observabilidade em produção, o fato de ter sido escrito por um agente é irrelevante. O critério não é a origem, é a evidência. P: Quanto tempo leva para ver resultado? R: Na primeira semana você já vê o efeito de escrever critérios de aceite verificáveis, independentemente de agentes. Os ganhos estruturais aparecem em um a dois ciclos.

Um convite para a prática deliberada

Conhecimento sem prática é entretenimento disfarçado de aprendizado. Este capítulo termina com um convite para a prática deliberada: escolha um artefato real do seu trabalho — uma spec, um teste, um release — e aplique deliberadamente um dos conceitos aqui descritos. Anote o antes e o depois. Repita por quatro semanas. No fim do mês, compare: o processo está mais previsível? O custo de contexto caiu? A estabilidade melhorou? Esse experimento pessoal, pequeno e mensurável, vale mais do que qualquer curso. É assim que o ciclo orientado a agentes deixa de ser um conceito que você explica para outras pessoas e se torna uma capacidade que você demonstra.

Próximos Passos

Você acabou de percorrer um dos capítulos centrais do SDLC AI-first. Se este conteúdo fez sentido para você, o próximo passo natural é continuar a jornada pelo livro completo, que aprofunda cada fase do ciclo: especificação executável, design de domínio, harness e agentes, verificação adversarial, entrega segura, aprendizado contínuo, economia de tokens e governança de maturidade.

Enquanto isso, aqui vão três ações concretas:

1. **Pratique em um projeto pequeno.** Nada substitui experimentar com as próprias mãos — escolha um repositório pessoal e aplique um dos conceitos deste capítulo.
2. **Formalize seu contrato.** Escreva o critério de aceite de uma tarefa real antes de delegá-la. É um exercício de cinco minutos que muda completamente a qualidade do resultado.
3. **Mensure seu processo.** Registre quanto tempo e quanto contexto cada fase consome. O que não é medido não pode ser melhorado.

O céu do software agora tem torres de controle. O próximo voo é seu.

Boa leitura e bons voos.

— Heverton Eduardo Peres.

